

What are global, protected and private attributes in Python?

- **Global variables** are public variables that are defined in the global scope. To use the variable in the global scope inside a function, we use the global keyword.

Ex:-

```
x = 10  # global variable
```

```
def update_global():  
    global x  # tell Python to use global x  
    x = x + 5
```

```
update_global()  
print(x)  # Output: 15
```

What are global, protected and private attributes in Python?

- **Protected attributes** are attributes defined with an underscore prefixed to their identifier eg. `_sara`. They can still be accessed and modified from outside the class they are defined in but a responsible developer should refrain from doing so.

Ex:-

```
class Employee:
```

```
    def __init__(self, name, salary):  
        self._salary = salary # protected attribute  
        self.name = name
```

```
emp = Employee("Ritu", 50000)
```

```
print(emp._salary) # ⚠️ works, but should be avoided
```

What are global, protected and private attributes in Python?

Private attributes are attributes with double underscore prefixed to their identifier eg. `__ansh`. They cannot be accessed or modified from the outside directly and will result in an `AttributeError` if such an attempt is made.

class Student:

```
def __init__(self, name, marks):  
    self.__marks = marks # private attribute  
    self.name = name
```

```
def get_marks(self): # getter method  
    return self.__marks
```

```
stu = Student("Dushyant", 95)
```

```
# print(stu.__marks) # ✗ AttributeError  
print(stu.get_marks()) # ✔ 95
```

```
# But still accessible with name-mangling:  
print(stu._Student__marks) # ⚠ 95
```

What is lambda in Python? Why is it used?

A **lambda function** is a **small anonymous function** in Python.

It can have **any number of arguments** but **only one expression**.

Why is it used?

- To create **short, throwaway functions** without formally defining them using `def`.
- Very useful in situations where a function is needed only once or for a short scope.
- Often used with **higher-order functions** like `map()`, `filter()`, `reduce()`, and `sorted()`.

Lambda Use case

1. Simple Lambda

```
square = lambda x: x * x  
print(square(5)) # Output: 25
```

2. Lambda with Multiple Arguments

```
add = lambda a, b: a + b  
print(add(10, 20)) # Output: 30
```

3. Using Lambda with `map()`

```
nums = [1, 2, 3, 4, 5]  
squares = list(map(lambda x: x**2, nums))  
print(squares) # Output: [1, 4, 9, 16, 25]
```

4. Using Lambda with `filter()`

```
nums = [10, 15, 20, 25, 30]  
even_nums = list(filter(lambda x: x % 2 == 0, nums))  
print(even_nums) # Output: [10, 20, 30]
```

What are Generators?

Generators are special functions in Python that **return an iterator** using the `yield` keyword instead of `return`.

They allow you to **generate values one at a time**, rather than computing and storing all values at once.

They are **memory-efficient** because values are produced **on-demand (lazy evaluation)**.

Why use Generators?

1. **Memory efficiency** → they don't store the entire result in memory.
2. **Faster performance** for large datasets.
3. **Useful in streaming/real-time data processing** where data is too large to fit in memory.

Generators

Normal function (returns a list - consumes memory):

```
def get_squares(n):  
    result = []  
    for i in range(n):  
        result.append(i*i)  
    return result  
  
print(get_squares(5))  # [0, 1, 4, 9, 16]
```

Generator (returns values one by one):

```
def get_squares_gen(n):  
    for i in range(n):  
        yield i*i  
  
gen = get_squares_gen(5)  
print(next(gen))  # 0  
print(next(gen))  # 1  
print(list(gen))  # [4, 9, 16]
```

Files

File is the collection of data that is available to a program. We can retrieve and use data stored in a file whenever we required.

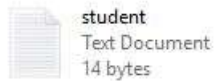
Advantages:-

- Stored Data is permanent unless someone remove it.
- Stored data can be shared.
- It is possible to update or remove the data.

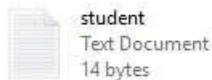
Type of Files

There are two type of files:-

Text File – It stores data in the form of characters. It is used to store characters and strings.



Binary File – It stores data in the form of bytes, a group of 8 bits each. It is used to store text, images, pdf, csv, video and audio.



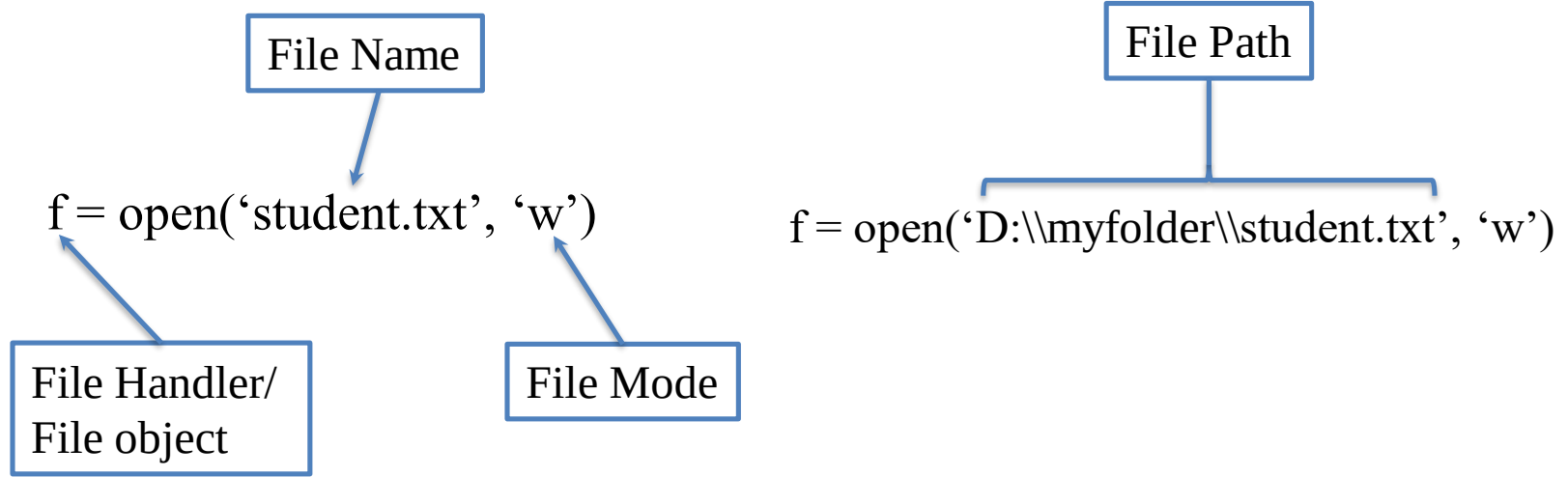
Text Mode and Binary Mode

Text Mode – A file opened in text mode, treats its contents as if it contains text strings of the str type.

When you get data from a text mode file, Python first decodes the raw bytes using either a platform-dependent encoding or, specified one.

Binary Mode – A file opened in Binary Mode, Python uses the data in the file without any decoding, binary mode file reflects the raw data in the file.

Opening a File



Text File Mode

Character	Meaning
r	Open for reading. The file pointer is positioned at the beginning of the file. If the file doesn't exist it will show FileNotFoundError.
w	Open for writing. If any data is already present in the file, it will overwrite the data. If the file doesn't exist it will create that file.
x	Open for exclusive creation with write. The specified file must not be available, if the specified file is available it will show error FileExistsError
a	Open for appending. The file pointer is positioned at the end of the file. It appends new data at the end of file. If the file does not exists it will create a new file for writing data.
r+	Open for reading and then writing
w+	Open for writing and then reading. It will overwrite existing data
a+	Open for appending then reading. It won't overwrite existing data

Binary File Mode

Character	Meaning
rb	Open for reading. The file pointer is positioned at the beginning of the file. If the file doesn't exist it will show FileNotFoundError.
wb	Open for writing. If any data is already present in the file, it will overwrite the data. If the file doesn't exist it will create that file.
xb	Open for exclusive creation with write. The specified file must not be available, if the specified file is available it will show error FileExistsError
ab	Open for appending. The file pointer is positioned at the end of the file. It appends new data at the end of file. If the file does not exist it will create a new file for writing data.
rb+	Open for reading and then writing
wb+	Open for writing and then reading. It will overwrite existing data
ab+	Open for appending then reading. It won't overwrite existing data

Closing a File

close() – This method is used to close, opened file.

Once we close the file, file object is deleted from the memory hence file will be no longer accessible unless we open it again.

If you don't explicitly close a file, Python's garbage collector will eventually destroy the object and close the open file for you, but the file may stay open for a while so You should always close opened file.

What will happened if we do not close opened file:-

- Data of the file may be corrupted or deleted.
- Memory utilized by the file is not freed it may cause of insufficient memory.

```
import csv
import json
import os

csv_file = "data.csv"
json_file = "data.json"

if os.path.exists(csv_file):
    data = []

    # --- Open CSV ---
    f = open(csv_file, mode='r')
    csv_reader = csv.DictReader(f)
    for row in csv_reader:
        data.append(row)
    f.close() # ✓ close CSV file

    # --- Write JSON ---
    f = open(json_file, mode='w')
    json.dump(data, f, indent=4)
    f.close() # ✓ close JSON file

    print(f"✓ {csv_file} successfully converted to {json_file}")
else:
    print(f"✗ File not found: {csv_file}")
```

Pickling

Pickling is a process of converting a class object into a byte stream so that it can be stored into a file. This is also called as object serialization.

We use *pickle* module to perform pickling and unpickling.

Function

`dump( )` – This function is used to perform the pickling. It returns the pickled representation of the object as a bytes object, instead of writing it to a file.

This method belongs to pickle module.

Syntax:-

```
import pickle
```

```
pickle.dump(object, file)
```

Unpickling

Unpickling is a process whereby byte stream is converted back into a class object. It is inverse operation of pickling. This is also called as de-serialization. Pickling and unpickling should be done using binary files since they support byte streams.

We use *pickle* module to perform pickling and unpickling.

Warning: The pickle module is not secure against erroneous or maliciously constructed data. Never unpickle data received from an untrusted or unauthenticated source.

Function

load() – This function is used to read an pickled object from a binary file and returns it into object. This method belongs to pickle module.

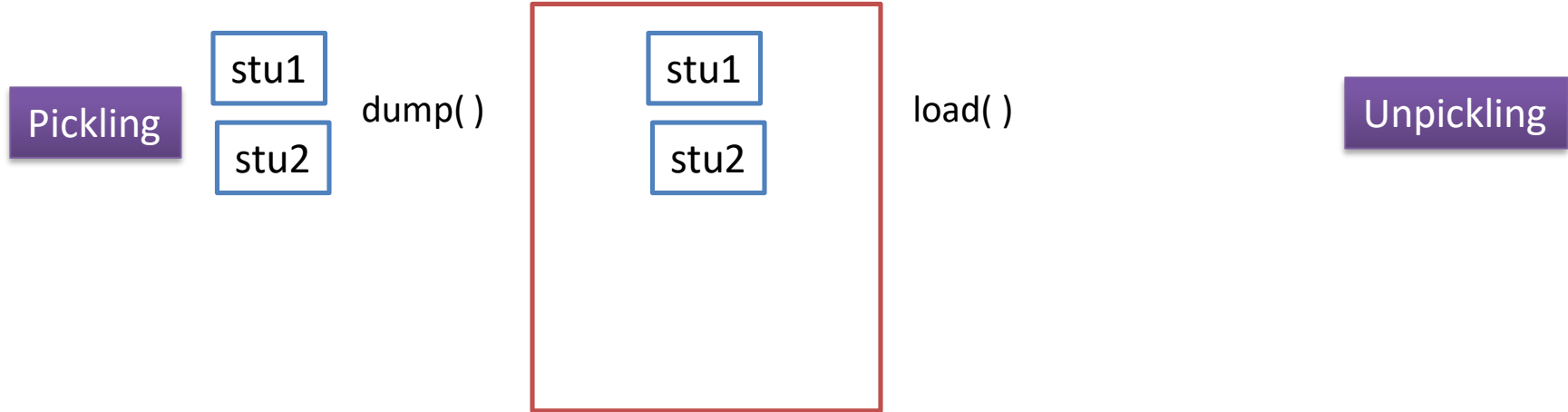
Syntax:-

```
import pickle
```

```
pickle.load(file)
```

Why do we need Pickling and Unpickling

When we store some structured data in the file and want to perform calculation that time we need pickling and unpickling.



```
import pickle, time
```

```
def expensive_function():
```

```
    time.sleep(3) # simulate heavy work
```

```
    return {"data": [1,2,3,4,5]}
```

```
# First run: compute & pickle
```

```
result = expensive_function()
```

```
with open("cache.pkl", "wb") as f:
```

```
    pickle.dump(result, f)
```

```
#Opens (or creates) cache.pkl in write-binary mode ("wb").
```

```
#Serializes (dump) result and
```

```
# Later run: load cached result
```

```
with open("cache.pkl", "rb") as f:
```

```
    cached_result = pickle.load(f)
```

```
#Opens cache.pkl in read-binary mode ("rb").
```

```
#Deserializes (load) the bytes back into a Python object → cached_result.
```

```
print("Loaded from cache:", cached_result)
```